

main_CNN.py Documentation

Overview

This project implements a Convolutional Neural Network (CNN) for binary soil classification using TensorFlow/Keras. The model distinguishes between soil types. The implementation includes data pre-processing, model training, evaluation, and feature visualisation using t-SNE.

Requirements

System Requirements

- Python 3.10.11 (downgrade required for TensorFlow/keras package)
- GPU support

Dependencies

In requirement.txt

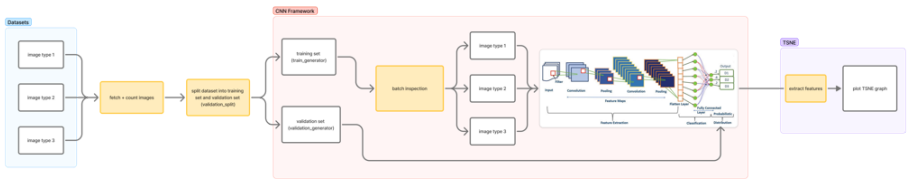
Installation

1. Clone repository.
2. Install dependencies.
3. Create folder and upload image dataset with clear labelling.
 - a. for example:

```
b. 1 GCS Training/                # Dataset directory
   2 |   | OA/                    # type 1 images
   3 |   |   | image1.jpg
   4 |   |   | image2.jpg
   5 |   |   | ...
   6 |   | OB0A/                  # type 2 images
   7 |   |   | image1.jpg
   8 |   |   | image2.jpg
   9 |   |   | ..
```

4. Update script with dataset directory path.
5. Change hyperparameters accordingly. Parameters are explained below.
6. Run code.

Code Architecture



Hyperparameters

Validation Split

```
1 validation_split = 0.30
```

Changes the split between training and test datasets. For this split, it's 70% training dataset and 30% (0.30) test dataset.

Model Output

```
1 tf.keras.layers.Dense(2, activation = 'softmax') #layers[13]
```

number 2 here represents the number of model outputs. Change it according to the number of classes. For example, if there are 10 classes of soil, the code will be:

```
1 tf.keras.layers.Dense(10, activation = 'softmax') #layers[13]
```

Epoch

```
1 epochs = 20
```

An epoch is one complete pass through the entire training dataset during the training process.

This changes the number of epochs or iterations that the machine will take.

If it is too low: the model is not accurate enough, underfitting occurs.

If it is too high: overfitting occurs, the machine performs really well during training, but poorly with new data.

An epoch of 20 is chosen because TBM dataset is limited in number and variety. By increasing the number of epochs, the machine is able to increase its accuracy despite limited data. However, this needs to be tuned once new data is obtained and a new function is given.

Computing TSNE Graph

Introduction to TSNE

Please read the articles below to gain a better understanding of TSNE.

[↗ Introduction to t-SNE: Nonlinear Dimensionality Reduction and Data Visualization](#)

[↗ Mastering Feature Extraction: PCA, t-SNE, and LDA in Machine Learning Part-2](#)

Choosing specific CNN layers for TSNE Feature Extraction

```
1 feature_extractor = tf.keras.Model(  
2     inputs = model.layers[0].input, #layer 0 used as input  
3     outputs = model.layers[-3].output #layer -3(third layer from the last) used as output  
4 )
```

Each layer number represents a different CNN layer (layer numbers are commented below). When applying t-SNE analysis, each CNN layer will generate its own unique visualization graph.

```
1 model = tf.keras.models.Sequential([  
2     tf.keras.layers.Conv2D(16, (3, 3), activation = 'relu', input_shape = input_shape), #layers[0]  
3     tf.keras.layers.MaxPooling2D(2, 2), #layers[1]  
4  
5     tf.keras.layers.Conv2D(32, (3, 3), activation = 'relu'), #layers[2]  
6     tf.keras.layers.MaxPooling2D(2, 2), #layers[3]  
7  
8     tf.keras.layers.Conv2D(64, (3, 3), activation = 'relu'), #layers[4]  
9     tf.keras.layers.MaxPooling2D(2, 2), #layers[5]  
10  
11    tf.keras.layers.Conv2D(128, (3, 3), activation = 'relu'), #layers[6]  
12    tf.keras.layers.MaxPooling2D(2, 2), #layers[7]  
13  
14    tf.keras.layers.Conv2D(128, (3, 3), activation = 'relu'), #layers[8]  
15    tf.keras.layers.MaxPooling2D(2, 2), #layers[9]  
16  
17    tf.keras.layers.Flatten(), #layers[10]  
18    tf.keras.layers.Dropout(0.5), #layers[11]  
19    tf.keras.layers.Dense(512, activation = 'relu'), #layers[12 or -3]  
20    tf.keras.layers.Dense(2, activation = 'softmax') #layers[13]
```

Graph Parameters

Signs You Need Parameter Changes:

- Clusters appear merged when they should be separate
- Points scattered randomly without clear structure
- Convergence warnings or poor separation between known classes
- Visualisation doesn't match expected data relationships

Recommendation: Start with these parameters, then adjust based on your specific data characteristics and visualisation quality.

- **perplexity:** Controls neighbourhood size. Higher values preserve global structure, lower values preserve local structure. Must match your data size and structure (too low = fragmented clusters, too high = merged clusters).
- **max_iter:** Increased from default (1000) to 2500 for better convergence on complex data. CNN features are complex; default 1000 iterations often insufficient for convergence.
- **init:** PCA initialization typically converges faster and more reliably than random initialisation
- **learning_rate:** Controls optimisation step size (default 200 usually sufficient). May need adjustment if clusters appear too compressed or scattered.
- **early_exaggeration:** Multiplier for attractive forces in early iterations to form distinct clusters. Helps separate overlapping classes in CNN feature space

```
1 n_samples = len(features)
2 perplexity = 45 #min(30, max(5, n_samples // 4)) #(max perplexity = 30, (n_samples x 0.25 = max, but minimum n_samples must be 5))
3
4 tsne = TSNE(
5     n_components = 2,
6     random_state = 42,
7     perplexity = perplexity,
8     #learning_rate = 500,
9     max_iter = 2500,
10    #early_exaggeration = 15,
11    init = 'pca'
```

